Szoftvertesztelés áttekintése

Dr. Micskei Zoltán
https://home.mit.bme.hu/~micskeiz

MŰEGYETEM 1782

BME VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR

Mesterséges Intelligencia és Rendszertervezés Tanszék

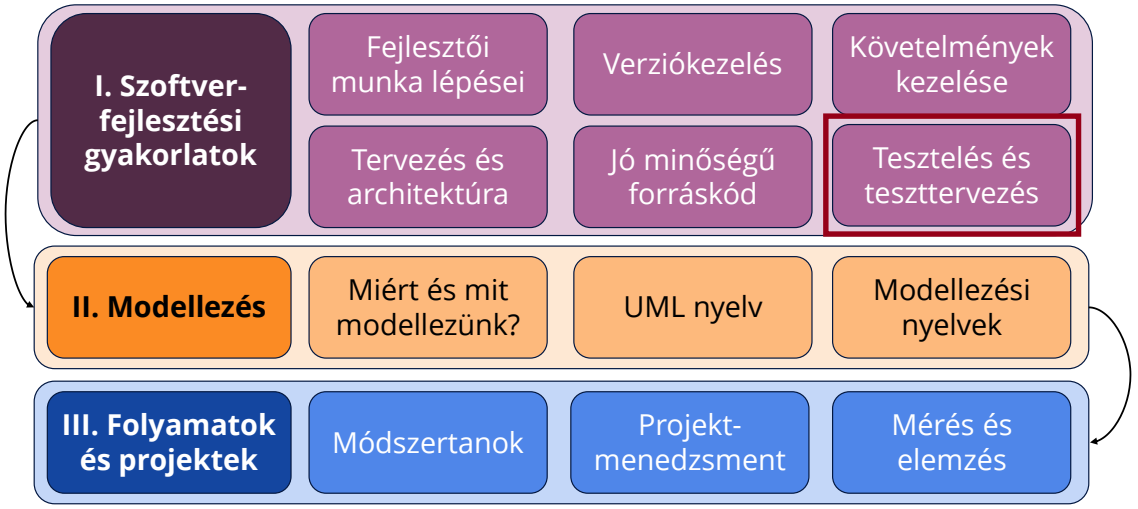
ftsrg Critical Systems Research Group

Tanulási eredmények

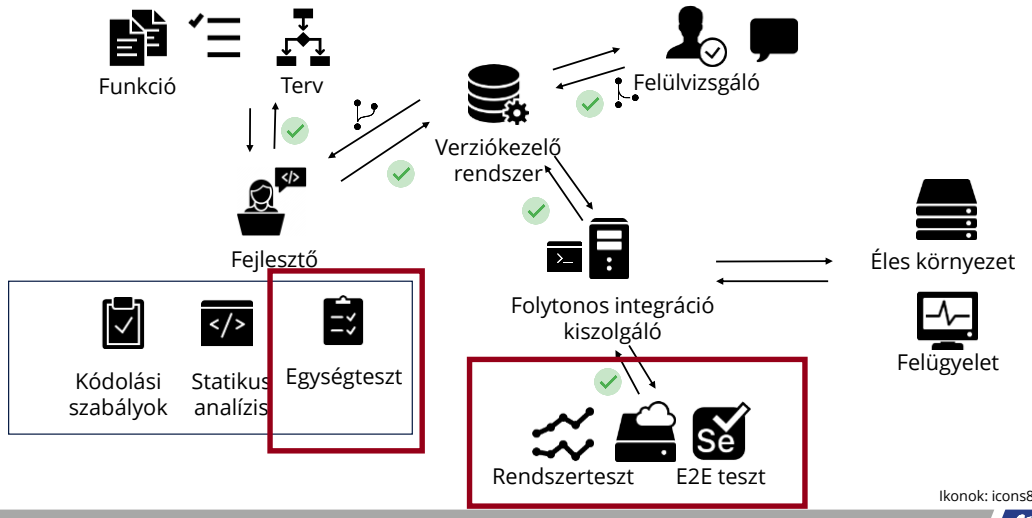
A tananyag elsajátítása után a hallgató képes lesz

- (K1) felidézni a szoftvertesztelés definícióját és céljait,
- (K2) összefoglalni a tipikus szoftvertesztelési folyamatot,
- (K2) megkülönböztetni a tesztelési szinteket és technikákat.

Szoftvertechnológia tantárgy témái



Tipikus fejlesztési munkafolyamat



Ikonok: icons8.com

Feladat: hogyan tesztelnénk?

Forrás: ECB

https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/eurofxref-graph-huf.en.html

Tesztelés

Definíció és célok

Ellenőrzési módszerek (emlékeztető)

Statikus

- **Mit**: bármilyen termék (dokumentáció, modell, kód)
- **Hogyan**: végrehajtás nélkül
- **Példa**: felülvizsgálat, statikus analízis

Dinamikus

- **Mit**: végrehajtható termék (kód, modell…)
- **Hogyan**: végrehajtva, futtatva
- **Példa**: szimuláció, tesztelés…

Tesztelés definíciója (1)

> A tesztelés egy tevékenység, aminek során a rendszert vagy komponenst meghatározott körülmények között végrehajtjuk, az eredményeket megfigyeljük és feljegyezzük, majd a rendszer vagy komponens valamilyen jellemzőjét kiértékeljük ennek alapján.
>
> Forrás: IEEE, "Systems and software engineering — Vocabulary," Standard 24765

„An activity in which a system or component is executed under specified conditions, the results are observed or recorded, and an evaluation is made of some aspect of the system or component.”

Egy másik IEEE definíció: „Testing is an activity performed for evaluating product quality, and for improving it, by identifying defects and problems.”

Tesztelés definíciója (2)

> Folyamat, amely az összes, szoftverfejlesztési életciklushoz kapcsolódó akár statikus, akár dinamikus tevékenységekből áll, amelyek kapcsolatban vannak egy komponens vagy rendszer, illetve a hozzá kapcsolódó munkatermékek tervezésével, előkészítésével és kiértékelésével. Célja, hogy
> – megállapítsa, a szoftvertermék teljesíti-e a meghatározott követelményeket,
> – megfelel-e a célnak és
> – megtalálja a hibákat.

Forrás: International Software Testing Qualifications Board (ISTQB)

V2: „The process consisting of all lifecycle activities, both static and dynamic, concerned with planning, preparation and evaluation of software products and related work products
- to determine that they satisfy specified requirements,
- to demonstrate that they are fit for purpose and
- to detect defects."

V3: „The process within the software development lifecycle that evaluates the quality of a component or system and related work products."

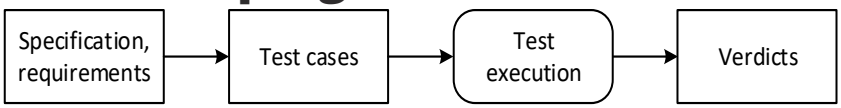
Tesztelés definíciója (3)

> A tesztelés a termék kiértékelésének folyamata, amelynek során felfedezéssel és kísérletezéssel tanulunk a termékről. Ezek magukban foglalják: kérdezés, tanulmányozás, modellezés, megfigyelés és következtetés, a kimenetek ellenőrzése stb.”

Forrás: James Bach, Michael Bolton. Exploratory Testing 3.0

“Testing is the process of evaluating a product by learning about it through exploration and experimentation, which includes: questioning, study, modeling, observation and inference, output checking, etc.”

Tesztelési alapfogalmak



- SUT: system under test
- Teszteset (test case)
  - bemeneti értékek, végrehajtási feltételek és elvárt eredmények halmaza, amelyeket egy konkrét célért fejlesztettek
- Tesztkészlet (test suite)
- Teszt-orákulum (test oracle)
  - Olyan elv vagy módszer, ami segít eldönteni a tesztelőnek, hogy sikeres volt-e a teszt
- Eredmény (verdict): pass / fail / error / inconclusive…

- **Test case** „set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement” (IEEE)
- **Test suite**: several test cases for a component or system under test
- **Teszt orákulum**: legegyszerűbb esetben megadjuk a teszteset tervezése során az elvárt kimenetet pontosan (pl. a specifikáció alapján). Gyakran ez nem lehetséges vagy túl költséges, ilyenkor lehet más típusú orákulumokat használni. Például összeomlások vagy kivételek detektálása (implicit oracle), vagy összehasonlítás korábbi verzióval vagy más implementációval (derived oracle).
- **Verdict**
  - **pass**: teszt sikeres volt
  - **fail**: nem az elvárt vagy az orákulumnak megfelelő eredményt kaptuk, a teszt valami hibára vagy eltérésre mutatott rá
  - **inconclusive**: akkor lehet például, ha a tesztesetben megadjuk, hogy milyen válaszüzenet a helyes, milyen válaszüzenet a hibás, és a konkrét tesztelés során ezektől eltérő választ kapunk
  - **error**: a teszt rendszerben keletkezett a hiba, így nem tudjuk eldönteni, hogy milyen volt a SUT válasza

Kimerítő (100%-os) tesztelés nem lehetséges / praktikus!

2 darab int paraméter?

2^32 * 2^32 = 2^64 = 1.8446744e+19
Másodpercenként 1 millió tesztesetet végrehajtva is ~585 000 évre lenne szükség.

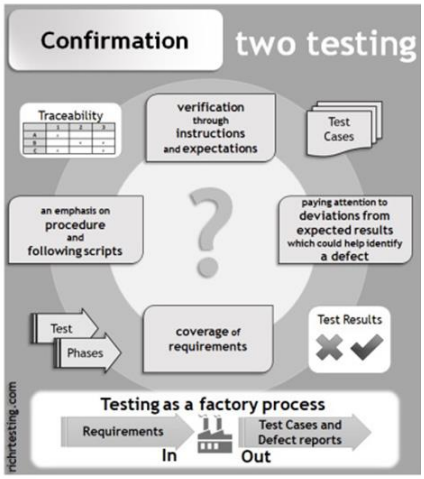
Megoldandó feladatok

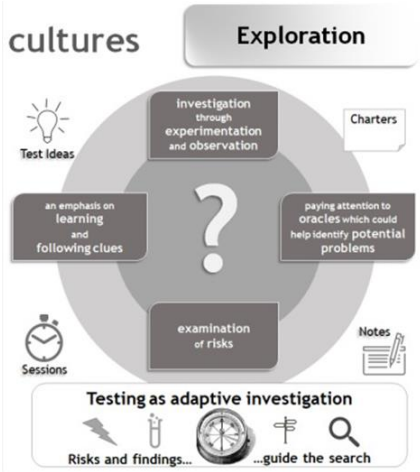
Tesztkiválasztás (test selection)	Orákulum problémája	Kilépési feltétel (exit criteria)
• Milyen bemeneti értékeket és adatok használjunk?	• Honnan lesz megbízható orákulum?	• Meddig folytassuk a tesztelést?

Tesztelés lehetséges céljai

és sok minden más ...

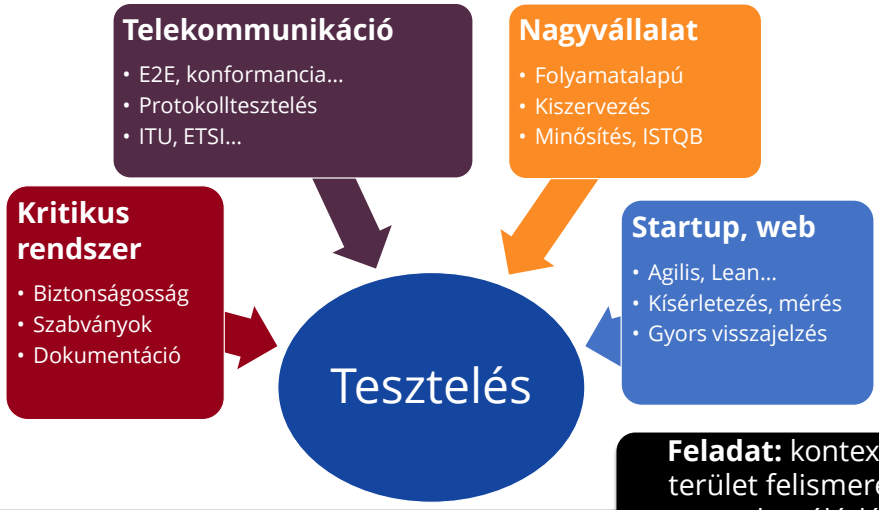
Megbizonyosodás és felderítés





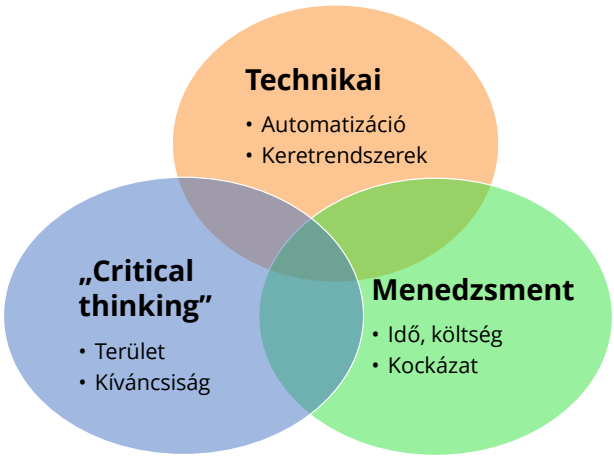
Source: Rich Rogers. "Confirmation and Exploration"

A tesztelés (is) kontextusfüggő!



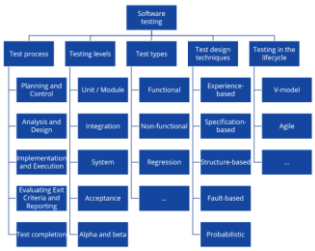
Feladat: kontextus és terület felismerése és adaptálódás

Nézőpontok a tesztelés során



Tesztelési folyamat

Lépések, feladatok, technikák

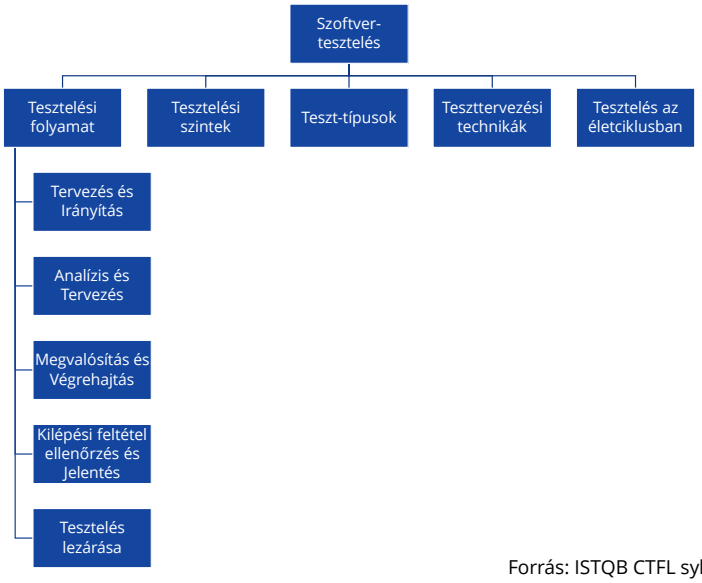


V&V: Verifikáció és Validáció

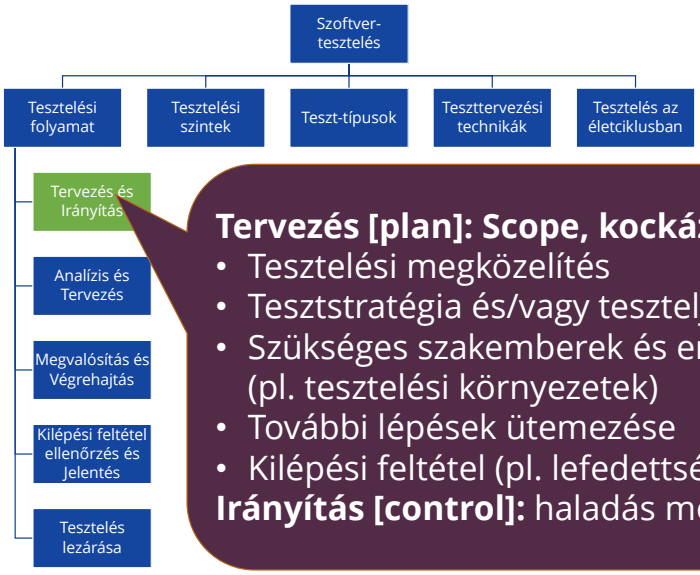
Verifikáció [Verification]	Validáció [Validation]
„Jól építem meg a rendszert?” [„Am I building the system right?”]	„A jó rendszert építem meg?” [„Am I building the right system?”]
A fejlesztési fázisok konzisztenciáját ellenőrizzük	A fejlesztés eredményét ellenőrizzük
A tervek/modellek/megvalósítás és a hozzájuk tartozó specifikáció konzisztenciája	Az elkészült rendszer és a felhasználói követelmények konzisztenciája
Objektív; (részben) automatizálható	Szubjektív; elfogadás ellenőrzése
Hibamodell: tervezési és implementációs hibák	Hibamodell: problémák a követelményekben

Bár ezek az általános meghatározások, az egyes szabványokban, eszközökben különböző módon használhatják; érdemes mindig ellenőrizni, hogy ki mit ért ezeken a fogalmakon.

Szoftver-
tesztelés
Tesztelési folyamat
Tesztelési szintek
Teszt-típusok
Teszttervezési technikák
Tesztelés az életciklusban



Forrás: ISTQB CTFL syllabus

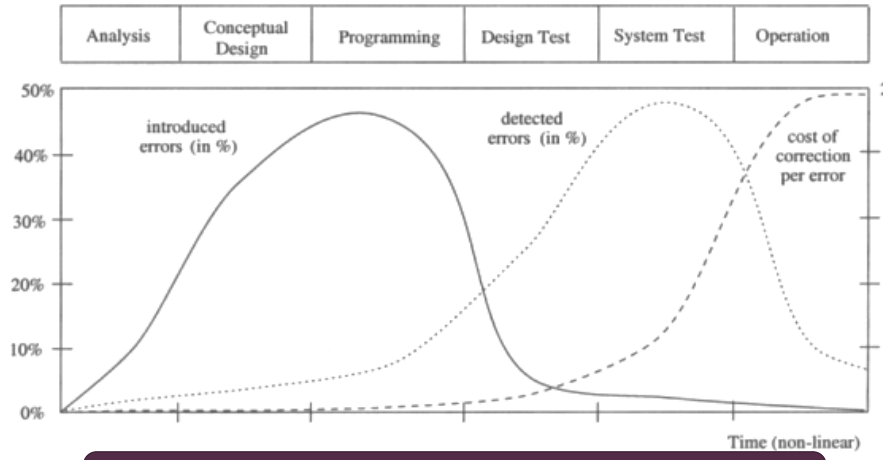


Tervezés [plan]: Scope, kockázat, célok

- Tesztelési megközelítés
- Tesztstratégia és/vagy teszteljárás
- Szükséges szakemberek és erőforrások (pl. tesztelési környezetek)
- További lépések ütemezése
- Kilépési feltétel (pl. lefedettség)

Irányítás [control]: haladás monitozása

Hibák eloszlása és javításuk költsége



Korai V&V jelentősen csökkenti a költségeket!

Alapesetben a hibák jelentős része jellemzően a korai fázisokban kerül be (pl. ellentmondásosak a követelmények, nincs jól meghatározva a rendszer célja, architektúra nem tudja teljesíteni az elvárásokat). Ezeknek a hibáknak a hatását minél később vesszük észre, minél később állunk neki az okokat megkeresni, majd javítani, annál nehezebb és költségesebb lesz.

Például ha a teljes rendszer elkészülte után végzett rendszertesztek során veszünk észre valami problémát, annak nagyon sok oka lehet. Sok idő és energia megtalálni a hiba okát, majd a javításához lehet, hogy több komponenst is át kell tervezni, újra letesztelni mindent, javítani a dokumentációt, újra egyeztetni több csapattal. Ha a hibát például az architektúra tervek átvizsgálása során észrevettük volna, akkor nagyon sok energiát és erőforrást lehetett volna megspórolni.

Emiatt a legtöbb fejlesztési folyamat hangsúlyt helyez a korai V&V tevékenységekre. Ennek ma egy divatos hívószava a „**shift left**” (told balra, azaz a fejlesztés korai fázisára az ellenőrzéseket).

Tesztelési stratégia [strategy]

- Ajánlások:
  - Milyen módszertant?
  - Milyen fajtájú teszteket?
  - Milyen eszközöket?
  - Ki fog tesztelni?
  - Kilépési feltétel?
  - Milyen dokumentáció?
- Leegyszerűsített példa:
  - Tesztvezérelt fejlesztés
  - Modul & rendszer
  - JUnit & GUI tesztelő
  - Fejlesztők & tesztmérnökök
  - Legalább 90%-os kódlefedettség & minden követelmény fedése
  - IEEE 29119-3 szerinti jelentés

További példákat lásd: Annex F, ISO 29119-3:2013

"Test strategy is an outline that describes the testing portion of the software development cycle. It is created to inform PM, testers and developers about some key issues of the testing process. This includes the testing objectives, method of testing, total time and resources required for the project and the testing environments."

Teszterv [test plan]

- Tesztelési stratégia leképezése a konkrét projektre
  - Célok [Test objectives]
  - Teszt tárgya [Test objects], tesztkörnyezet
  - Szereplők, erőforrások
  - Ütemezés

- Tesztelési fázisok meghatározása
  - Fázisok hossza
  - Kilépési feltétel
  - Tesztelés minőségének meghatározása

„teszterv (test plan): a teszt hatáskörét, megközelítését, erőforrásait valamint a tevékenységek tervezett ütemezését tartalmazó dokumentum. Ezen kívül meghatározza a tesztelemeket, a tesztelendő funkciókat, feladatokat, a tesztet végrehajtó személyek függetlenségét, a tesztkörnyezetet, a műszaki teszttervezési technikákat, a belépési és kilépési feltételeket, valamint kockázatokat. A teszttervezési folyamat meghatározó dokumentuma (IEEE 829 alapján)” [HTB Glossary]

Teztterv felépítése: ISO 29119-3:2013(E)

a) Context of the testing:
   - i) Project/Test sub-process
   - ii) Test item(s)
   - iii) Test scope
   - iv) Assumptions and constraints
   - v) Stakeholders

b) Testing communication

c) Risk register:
   - i) Product risks
   - ii) Project risks

d) Test strategy:
   - i) Test sub-processes
   - ii) Test deliverables
   - iii) Test design techniques
   - iv) Test completion criteria
   - v) Metrics to be collected
   - vi) Test data requirements
   - vii) Test environment requirements
   - xi) Retesting and regression testing
   - xii) Suspension and resumption criteria
   - xiii) Deviations from the Organizational Te

e) Testing activities and estimates

f) Staffing:
   - i) Roles, activities, and responsibilities
   - ii) Hiring needs
   - iii) Training needs

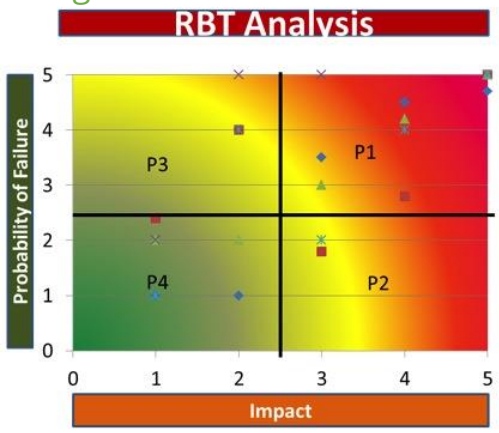
g) Schedule

A ISO/IEC/IEEE 29119-3:2013(E) szabvány függelékeiben vannak jó példák, hogy hogyan lehet ehhez hasonló teszt tervet megadni tradicionális vagy agilis környezetben.

Kockázatalapú tesztelés [risk-based]

Kockázat szintje = hatás * valószínűség

- P1 – Critical Impact: Must be tested; ideal candidate for automation (candidate for automated smoke testing)
- P2 – High Impact: Should be tested
- P3 – Medium Impact: Can be tested if budget and schedule permits
- P4 – Low Impact: May not be tested; no impact on application and no need of automation

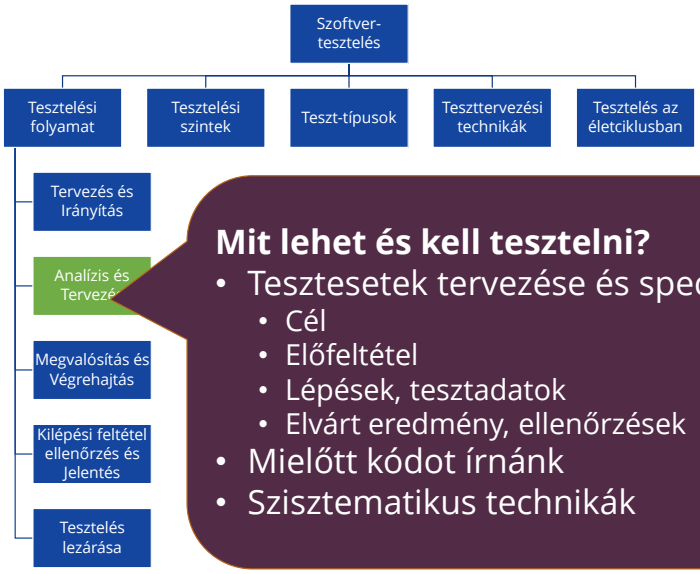


Forrás: Risk-Based Testing: Test Only What Matters

Hasonlóan a követelmények prioritásának meghatározásához, a tesztelést is vezetheti a kockázatalapú gondolkozás.

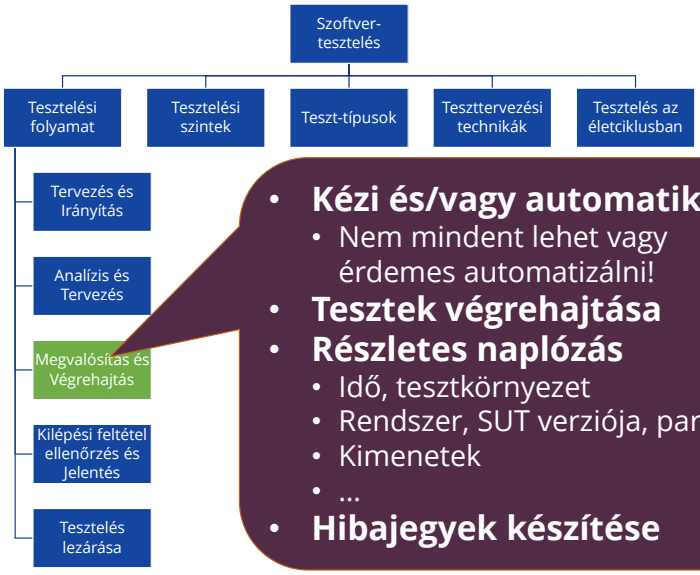
Mivel nem tudunk mindent tesztelni, ezért arra érdemes koncentrálni, aminek kritikus hatása lehet és nagyobb a valószínűsége, hogy bekötvetkezik.

Például, ha egy új webes rendszert fejlesztünk, és nincs nagy tapasztalatunk a fizetési megoldások (bankkártya, azonnali átutalás…) integrálásával, akkor ennek az ellenőrzése, tesztelése kiemelt figyelmet érdemel. Ugyanakkor a felhasznált, modern webes keretrendszer miatt lehet, hogy a régi böngészőket használó felhasználóknak majd nem működik minden jól, de úgy mértük fel, hogy ez csak a felhasználók nagyon kis részét érinti, így most nem erre koncentrálunk.

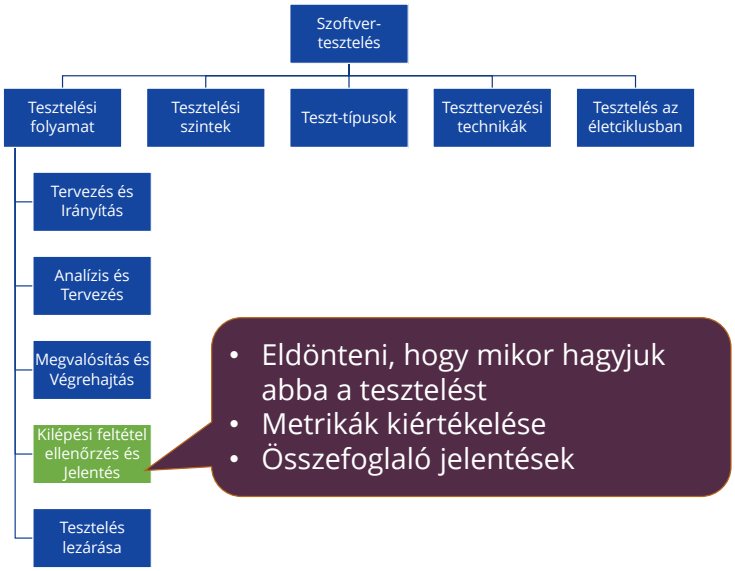


Mit lehet és kell tesztelni?

- Tesztesetek tervezése és specifikálása
  - Cél
  - Előfeltétel
  - Lépések, tesztadatok
  - Elvárt eredmény, ellenőrzések
- Mielőtt kódot írnánk
- Szisztematikus technikák

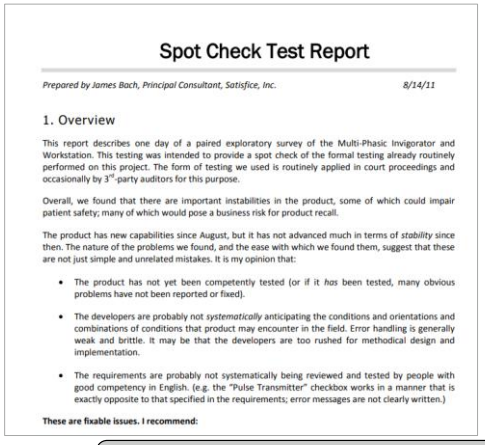


- **Kézi és/vagy automatikus**
  - Nem mindent lehet vagy érdemes automatizálni!
- **Tesztek végrehajtása**
- **Részletes naplózás**
  - Idő, tesztkörnyezet
  - Rendszer, SUT verziója, paraméterei
  - Kimenetek
  - …
- **Hibajegyek készítése**



- Eldönteni, hogy mikor hagyjuk abba a tesztelést
- Metrikák kiértékelése
- Összefoglaló jelentések

Példa: Tesztelési jelentés



Spot Check Test Report

Prepared by James Bach, Principal Consultant, Satisfice, Inc. *8/14/11*

1. Overview

This report describes one day of a paired exploratory survey of the Multi-Phasic Invigorator and Workstation. This testing was intended to provide a spot check of the formal testing already routinely performed on this project. The form of testing we used is routinely applied in court proceedings and occasionally by 3rd party auditors for this purpose.

Overall, we found that there are important instabilities in the product, some of which could impair patient safety; many of which would pose a business risk for product recall.

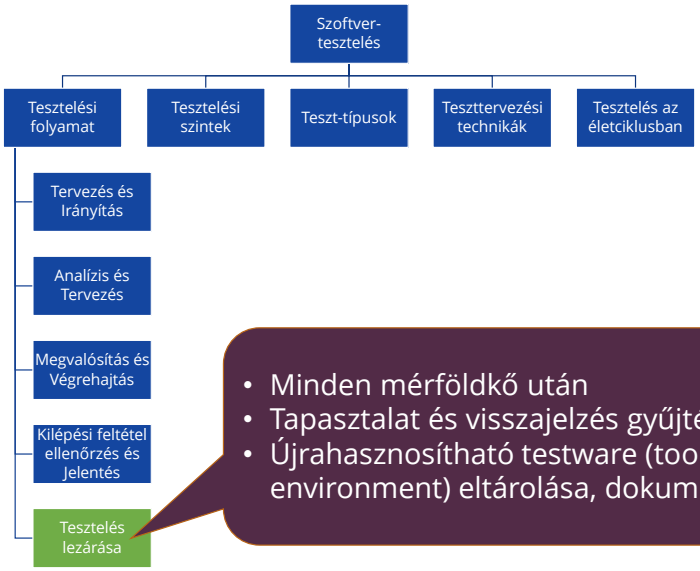
The product has new capabilities since August, but it has not advanced much in terms of *stability* since then. The nature of the problems we found, and the ease with which we found them, suggest that these are not just simple and unrelated mistakes. It is my opinion that:

- The product has not yet been competently tested (or if it *has* been tested, many obvious problems have not been reported or fixed).
- The developers are probably not *systematically* anticipating the conditions and orientations and combinations of conditions that product may encounter in the field. Error handling is generally weak and brittle. It may be that the developers are too rushed for methodical design and implementation.
- The requirements are probably not systematically being reviewed and tested by people with good competency in English. (e.g. the "Pulse Transmitter" checkbox works in a manner that is exactly opposite to that specified in the requirements; error messages are not clearly written.)

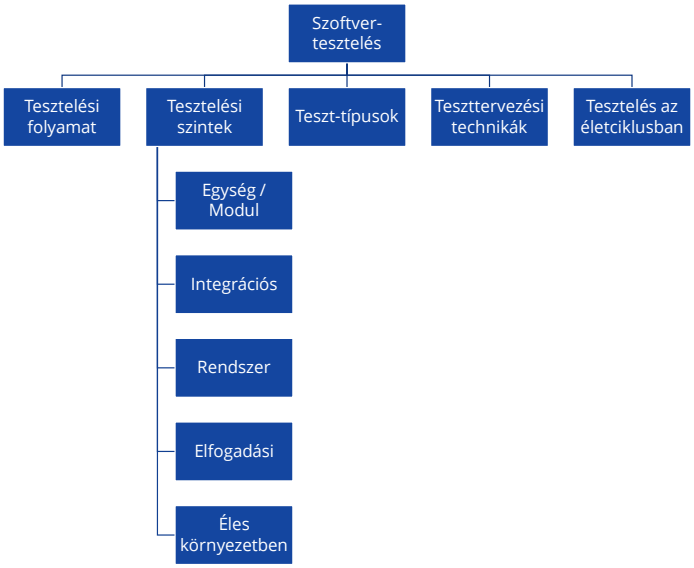
These are fixable issues. I recommend:

- Overview: vezetői összefoglaló
  - Fő üzenet (értékelés)
  - Legfontosabb problémák
  - Javaslatok
- Tesztelés bemutatása
  - Pontosan mit csináltunk
- Részletek
  - Súlyos problémák tételesen
  - További problémák

https://www.developsense.com/resources/ReportingExamples/mpim-report.pdf

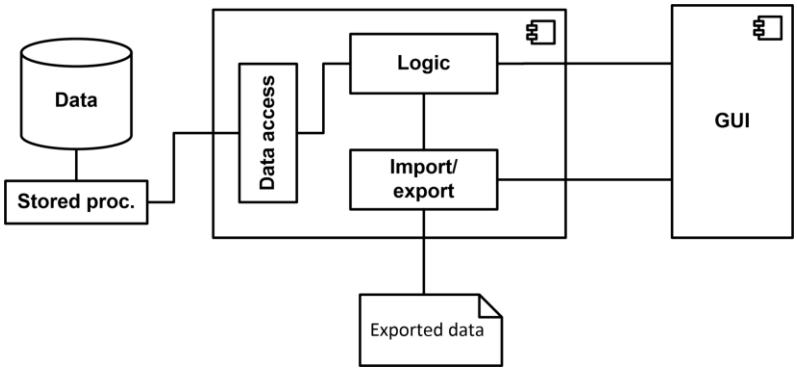


- Minden mérföldkő után
- Tapasztalat és visszajelzés gyűjtése
- Újrahasznosítható testware (tools, environment) eltárolása, dokumentálása

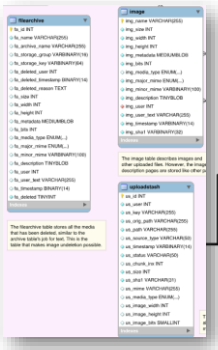


- Egységtesztelés / modultesztelés / komponenstesztelés [unit / module / component testing]: az önálló szoftver vagy hardver egységek/modulok/komponensek önálló a többitől független, különálló tesztelése.
- Integrációs tesztelés [integration testing]: „Tesztszint, mely a komponensek vagy rendszerek egymás közötti kölcsönhatásaira fókuszál.” [ISTQB]
- Rendszertesztelés [system testing]: „Tesztszint, mely a rendszer egészével szemben meghatározott követelményeknek való megfelelőséget vizsgálja.” [ISTQB]
- Elfogadási tesztelés [acceptance testing]: „Tesztszint, mely annak meghatározására fókuszál, hogy a rendszer elfogadható-e.”
- Tesztelés éles környezetben [Testing is production]
- A/B testing: „Tesztmegközelítés, amelyben két komponenst vagy rendszert statisztikailag értékelnek ki annak érdekében, hogy megállapítsák melyik teljesít jobban meghatározott feltételek között.” [ISTQB]

Példa: tesztelési szintek (mit/hogyan?)



Példa: tesztelési szintek (mit/hogyan?)

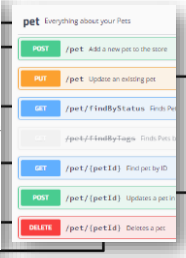


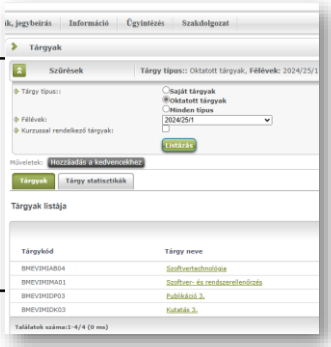
```
public class StationUtil
{

    public static boolean isLastStopOfALine(Station station)
    {

        List<Stop> stops = station.getStops();
        final int nStops = stops.size();
        for (int i = 0; i < nStops; i++) {
            Stop stop = stops.get(i);
            Line line = stop.getLine();
            if (line.isCircular()) {
                continue;
            }
            List<Stop> lineStops = line.getStops();
            if (stop == lineStops.get(0)
                    || stop == lineStops.get(lineStops.size() -
                    return true;
            }
        }
        return false;
    }

    public static Coordinate location(Stop stop)
    {
        if (stop.getLocation() != null) {
            return stop.getLocation();
        }
        return stop.getStation().getLocation();
    }
```





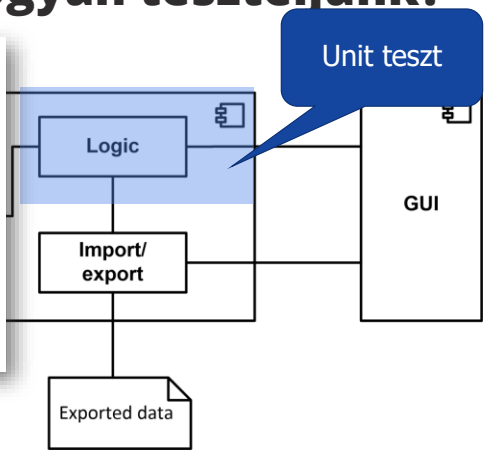
	A	B	C	E	G	H
1	Név	Neptunkód	Képzés	Felvételek	Vizsgára	Várólista
2	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem
3	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem
4	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem
5	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem
6	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem
7	Példa Név	NEPTUN	mérnökinformatikus	1	Igen	Nem

Példa: mit/hol/hogyan teszteljünk?

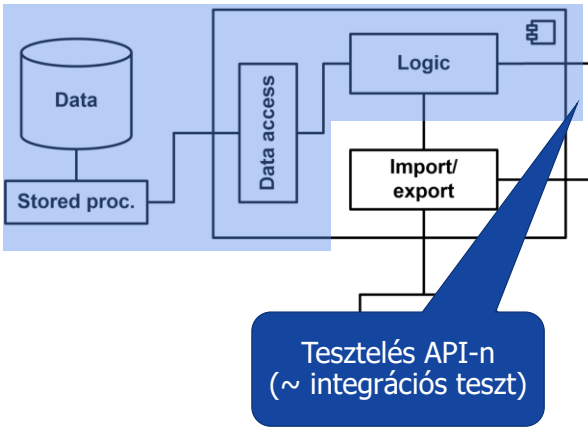
```
public class ListTest{
    @Test public void add_EmptyList_Success(){
        // Arrange
        list = new List();

        // Act
        list.Add(1);

        // Assert
        assertEquals(1, list.getSize());
    }
```



Példa: mit/hol/hogyan teszteljünk?



For API Testing

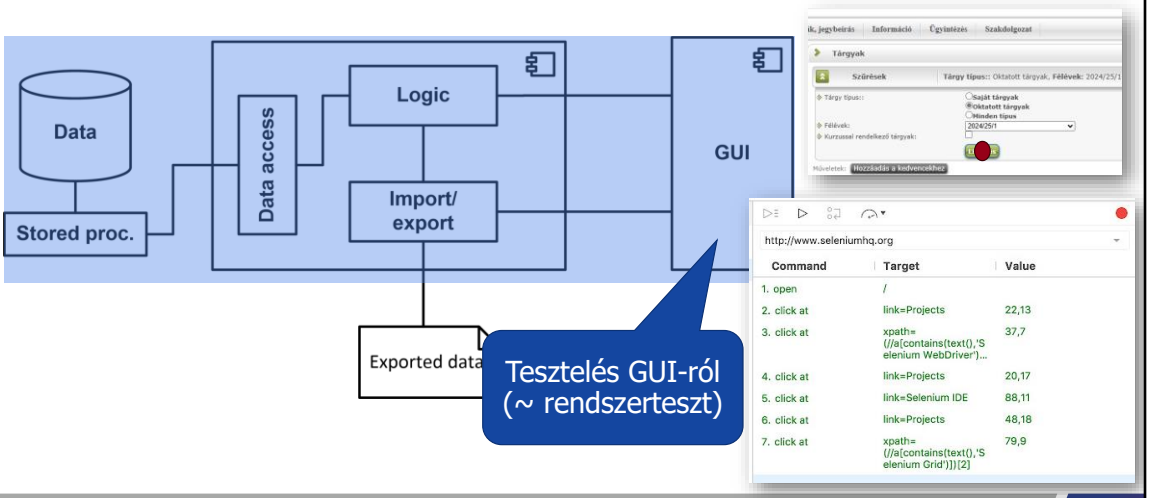
```
Scenario: create and retrieve a cat

Given url 'http://myhost.com/v1/cats'
And request { name: 'Billie' }
When method post
Then status 201
And match response == { id: '#notnull'

Given path response.id
When method get
Then status 200
```

Forrás: Karate Labs

Példa: mit/hol/hogyan teszteljünk?



Különbség az egység és a többi szint között

Egységtesztek [unit test]

- Néhány metódus hívása + paraméterek állítása
- Jobbára tesztkódból
- Legtöbbször az IDE-ben végrehajtva

Integrációs+ tesztek

- Metódushívások és paraméterek csak kis része
- Környezet beállítása sokkal nagyobb feladat
- Fájlok, adatbázisok, hálózat, jelszavak…
- Rengeteg dolog romolhat el a futtatáskor

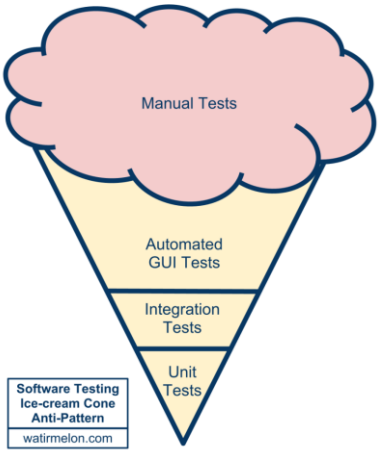
Különbség a szintek között

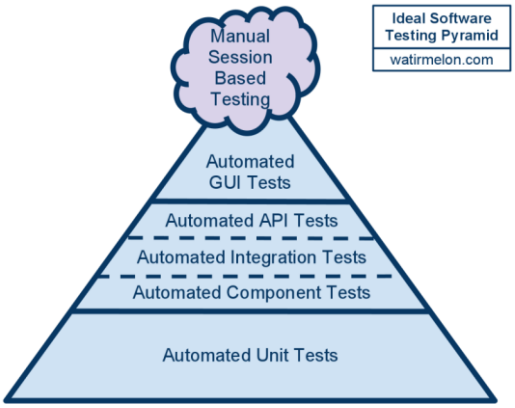
How Google Tests Software könyv ajánlásai:

(small ~ unit, medium ~ integration, large ~ sytem)

	Small	Medium	Large
Javasolt futási idő	< 100 ms	< 1 sec	minél gyorsabban
Időkorlát (kill)	1 perc	5 perc	1 óra
Erőforrás	**Small**	**Medium**	**Large**
Hálózat (socket)	Mocked	csak localhost	Igen
Adatbázis	Mocked	Igen	Igen
Fájl hozzáférés	Mocked	Igen	Igen
Rendszerhívás	Nem	Nem javasolt	Igen
Több szál	Nem javasolt	Igen	Igen
Sleep utasítás	Nem	Igen	Igen
Rendszer tulajdonságai	Nem	Igen	Igen

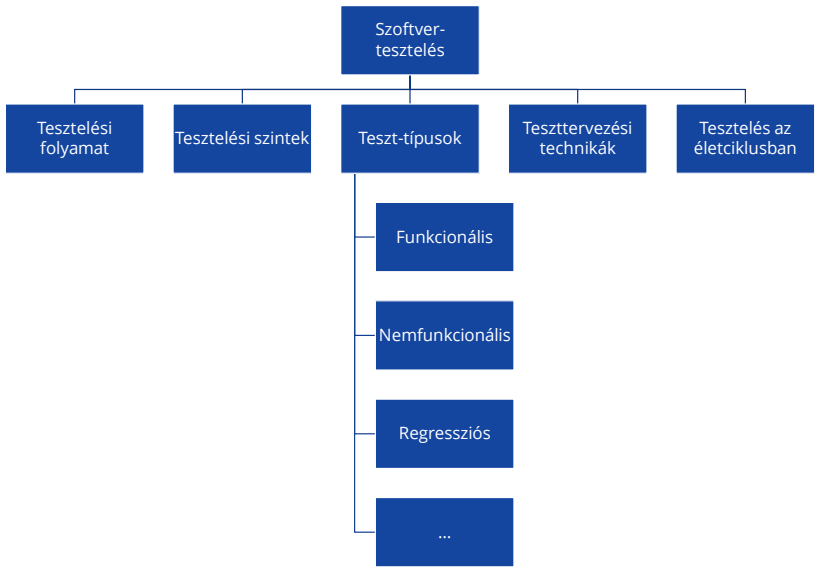
Tesztelési piramis



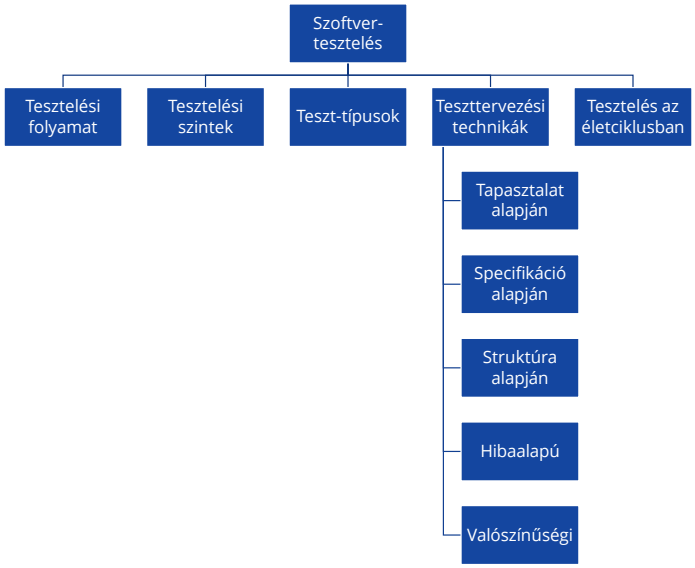


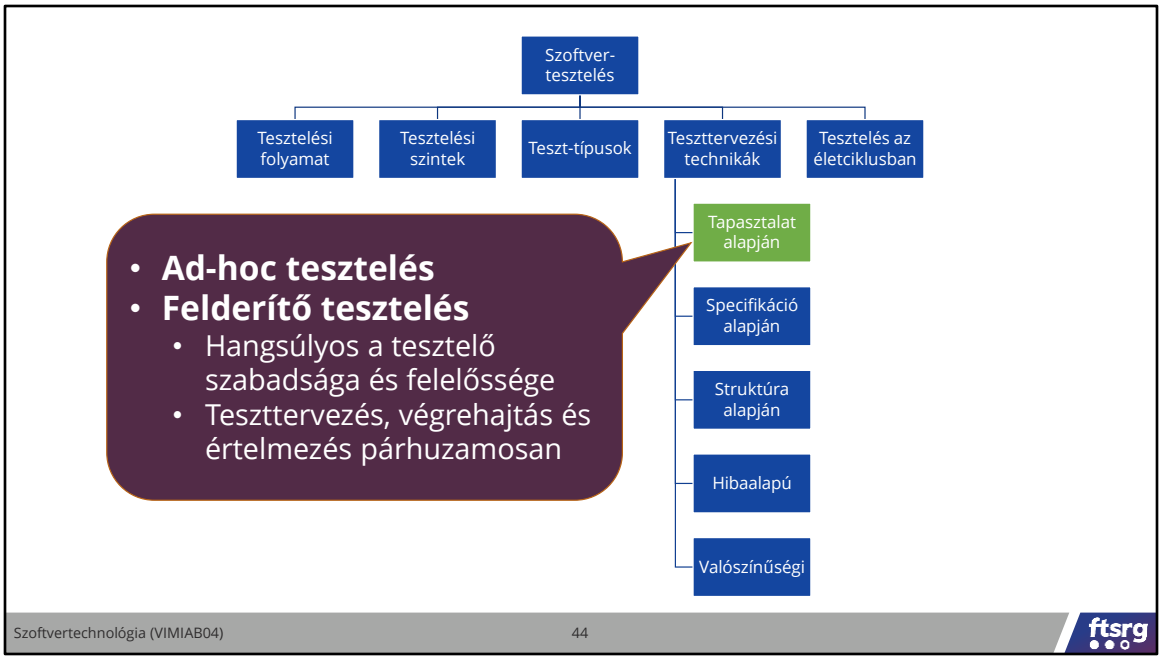
Forrás: Alister Scott

Lásd még: Mike Cohn, Martin Fowler...

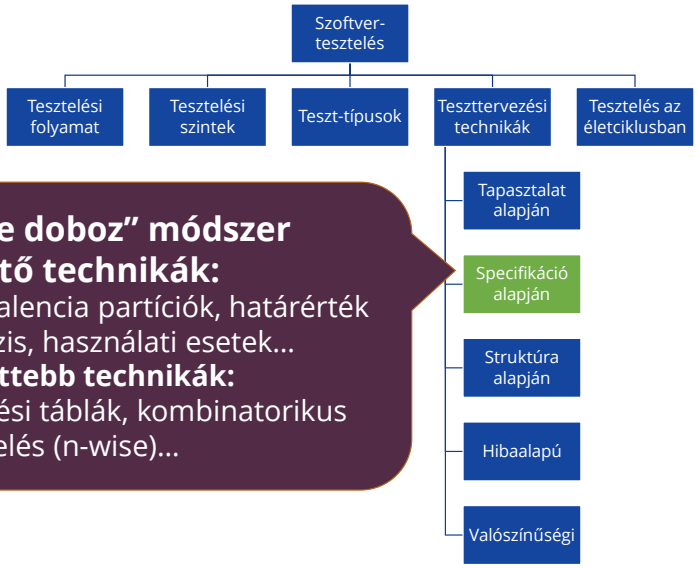


Regression testing: „testing performed following modifications to a test item or to its operational environment, to identify whether failures in unmodified parts of the test item occur” (IEEE)

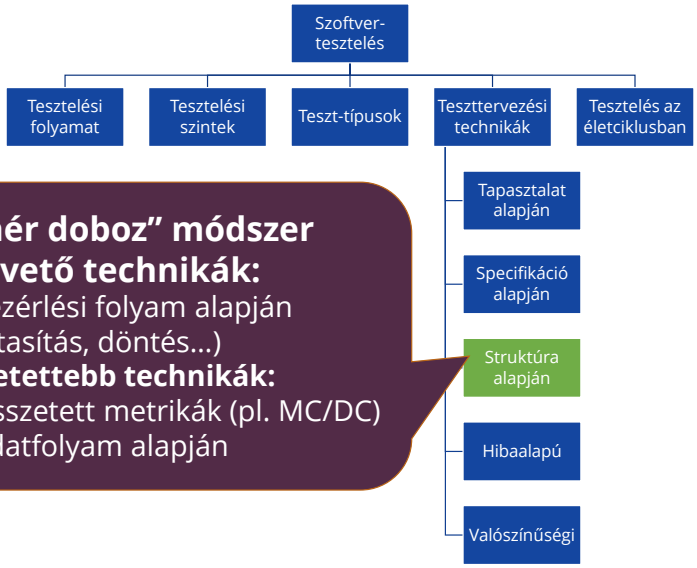

Szoftver-
tesztelés
Tesztelési folyamat
Tesztelési szintek
Teszt-típusok
Teszttervezési technikák
Tesztelés az életciklusban
Tapasztalat alapján
Specifikáció alapján
Struktúra alapján
Hibaalapú
Valószínűségi



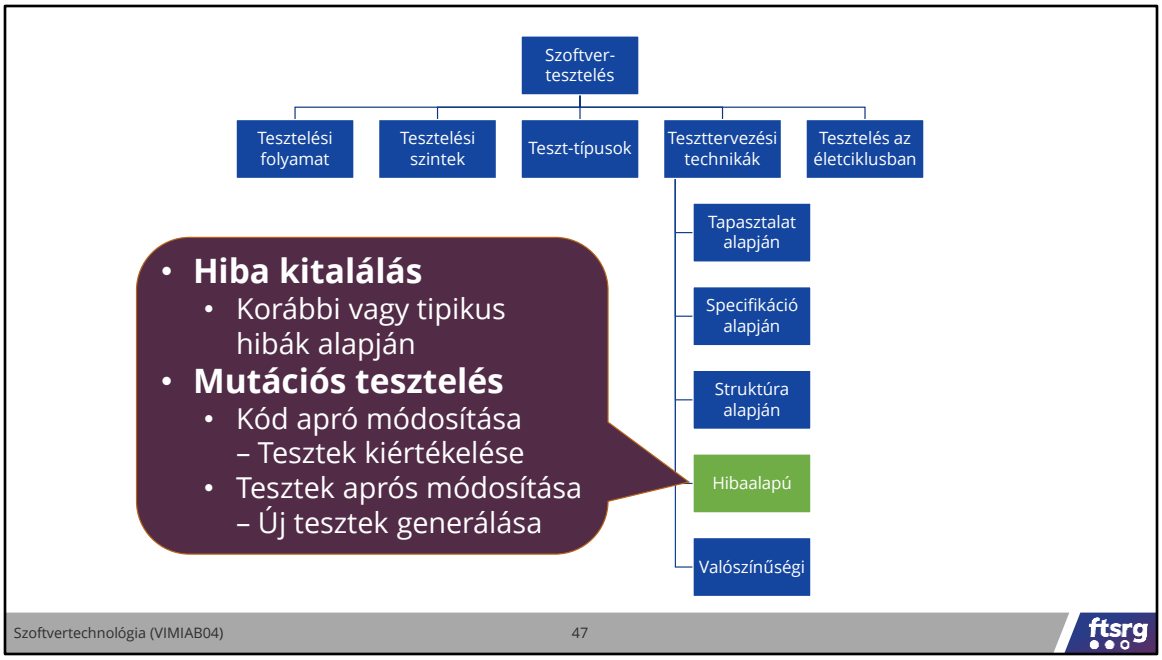
Felderítő tesztelés [Exploratory testing]



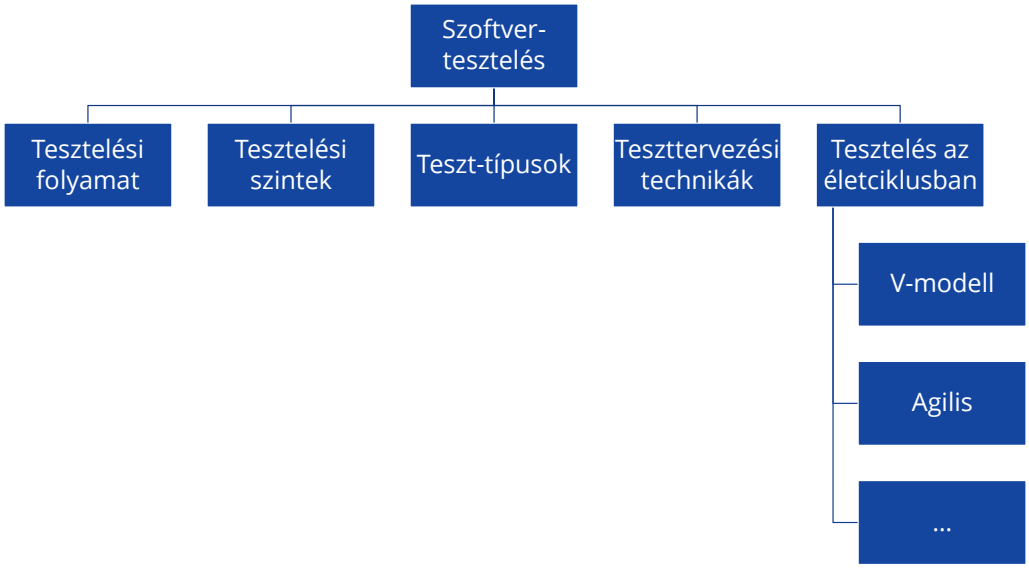
- **„Fekete doboz” módszer**
- **Alapvető technikák:**
  - Ekvivalencia partíciók, határérték analízis, használati esetek…
- **Összetettebb technikák:**
  - Döntési táblák, kombinatorikus tesztelés (n-wise)…



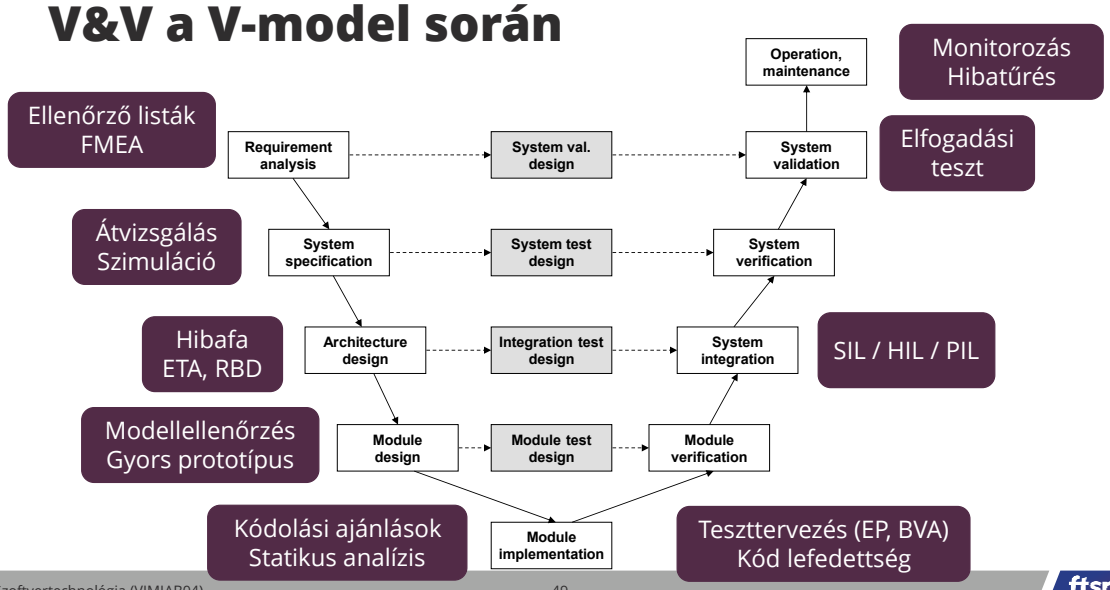
- **„Fehér doboz” módszer**
- **Alapvető technikák:**
  - Vezérlési folyam alapján (utasítás, döntés...)
- **Összetettebb technikák:**
  - Összetett metrikák (pl. MC/DC)
  - Adatfolyam alapján

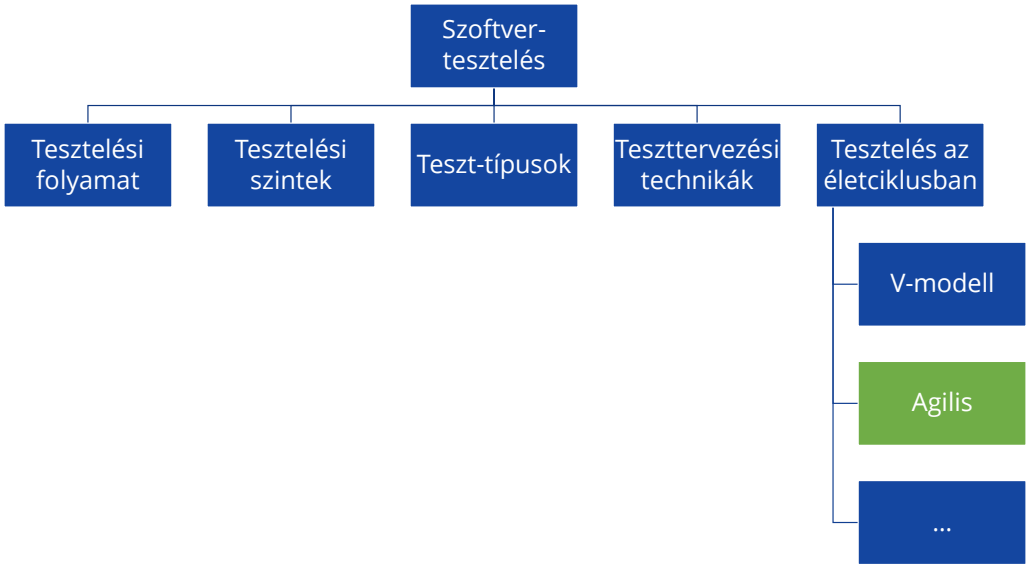


mutation testing. (1) testing methodology in which two or more program mutations are executed using the same test cases to evaluate the ability of the test cases to detect differences in the mutations

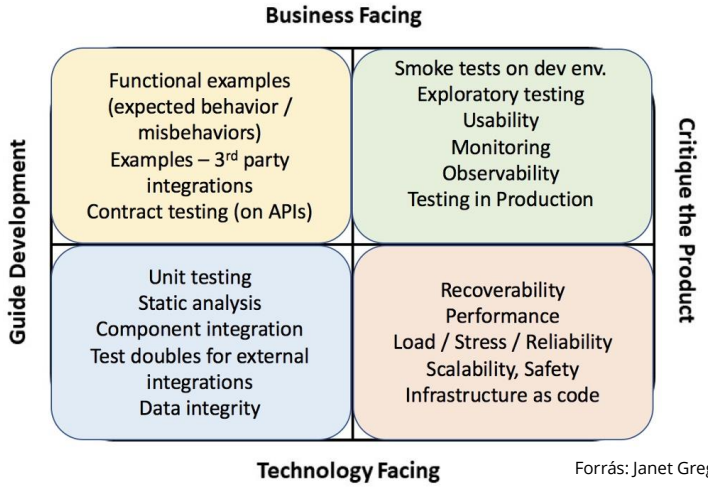

Szoftver-
tesztelés
Tesztelési folyamat
Tesztelési szintek
Teszt-típusok
Teszttervezési technikák
Tesztelés az életciklusban
V-modell
Agilis
...

V&V a V-model során



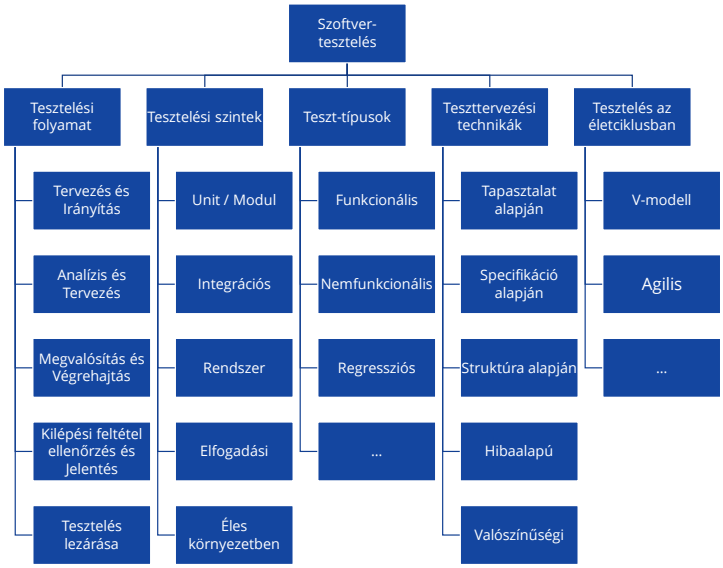

Szoftver-
tesztelés
Tesztelési folyamat
Tesztelési szintek
Teszt-típusok
Teszttervezési technikák
Tesztelés az életciklusban
V-modell
Agilis
...

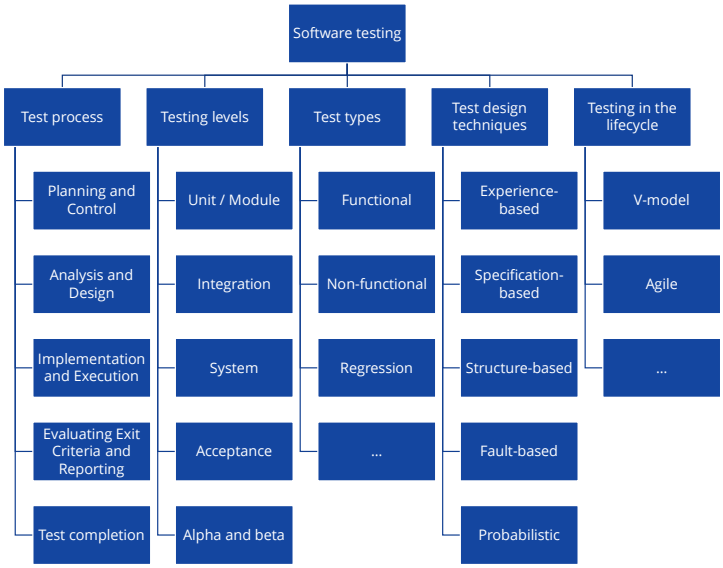
Agilis tesztelési kvadránsok



Forrás: Janet Gregory, Lisa Crispin. Agile Testing

How do all these verification fit together? What are their different purposes?
The agile testing quadrant offers a mental framework to assign a role to each of these techniques.


Szoftver-tesztelés
Tesztelési folyamat
Tesztelési szintek
Teszt-típusok
Teszttervezési technikák
Tesztelés az életciklusban
Tervezés és Irányítás
Analízis és Tervezés
Megvalósítás és Végrehajtás
Kilépési feltétel ellenőrzés és Jelentés
Tesztelés lezárása
Unit / Modul
Integrációs
Rendszer
Elfogadási
Éles környezetben
Funkcionális
Nemfunkcionális
Regressziós
...
Tapasztalat alapján
Specifikáció alapján
Struktúra alapján
Hibaalapú
Valószínűségi
V-modell
Agilis
...


Software testing
Test process
Testing levels
Test types
Test design techniques
Testing in the lifecycle
Planning and Control
Analysis and Design
Implementation and Execution
Evaluating Exit Criteria and Reporting
Test completion
Unit / Module
Integration
System
Acceptance
Alpha and beta
Functional
Non-functional
Regression
...
Experience-based
Specification-based
Structure-based
Fault-based
Probabilistic
V-model
Agile
...

Összefoglalás

Összefoglalás

Tesztelési alapfogalmak

Specification, requirements → Test cases → Test execution → Verdicts

- SUT: system under test
- Teszteset (test case)
  - bemeneti értékek, végrehajtási feltételek és elvárt eredmények halmaza, amelyeket egy konkrét célért fejlesztettek
- Tesztkészlet (test suite)
- Teszt-orákulum (test oracle)
  - Olyan elv vagy módszer, ami segít eldönteni a tesztelőnek, hogy sikeres volt-e a teszt
- Eredmény (verdict): pass / fail /error / inconclusive...

Megoldandó feladatok

Tesztkiválasztás (test selection)	Orákulum problémája	Kilépési feltétel (exit criteria)
• Milyen bemeneti értékeket és adatok használjunk?	• Honnan lesz megbízható orákulum?	• Meddig folytassuk a tesztelést?

Különbség az egység és a többi szint között

Egységtesztek [unit test]
- Néhány metódus hívása + paraméterek állítása
- Jobbára tesztkódból
- Legtöbbször az IDE-ben végrehajtva

Integrációs+ tesztek
- Metódushívások és paraméterek csak kis része
- Környezet beállítása sokkal nagyobb feladat
- Fájlok, adatbázisok, hálózat, jelszavak...
- Rengeteg dolog romolhat el a futtatáskor